

C++_QT_Day1 AI 사용 보고서, 2026406041 김도훈

사용한 LLM은 클로드 Sonnet 5입니다.

과제 1 AI 사용 내용

vector로 배열을 만들면 나중에 배열의 인덱스에 접근할 때 어떻게 해?

일반 배열이랑 거의 똑같아요. `[]` (대괄호)로 접근하면 됩니다.

기본 접근

```
cpp
vector<int> arr = {10, 20, 30, 40, 50};

cout << arr[0] << endl; // 10 (첫 번째 원소)
cout << arr[2] << endl; // 30 (세 번째 원소)

arr[1] = 100; // 두 번째 원소 값 변경 -> {10, 100, 30, 40, 50}
```

인덱스는 배열처림 0부터 시작합니다.

방금 만든 클래스 예제에서는

```
cpp
holder.array[i] // 객체, 벡터이름[인덱스]
```

이렇게 객체 접근(`[]`)이랑 인덱스 접근(`[]`)을 같이 쓰면 됩니다.

안전하게 접근하고 싶으면 `.at()`

```
cpp
cout << arr.at(2) << endl; // 30
```

`[]` 랑 결과는 똑같은데 차이가 하나 있어요.

방식	범위를 벗어난 인덱스 접근 시
<code>arr[10]</code>	정의되지 않은 동작 (프로그래머 이상하게 동작하거나 크래시)
<code>arr.at(10)</code>	예외(exception)를 던져줌 -> 에러 원인 파악이 쉬움

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

class ci{
    // ...
}
```

이건 버그가 아니라 `cout`의 기본 출력 방식 때문이에요. 계산 자체는 정상입니다.

확인

`((double)sum / num)` 연산 자체는 문제없이 실수로 계산되고 있어요. 다만 결과값이 딱 1.0처럼 소수점 이하가 전부 0인 경우, `cout`이 기본적으로 불필요한 소수점 0을 자동으로 생략하고 출력합니다.

예를 들어 평균이 2.57이면 `2.57`로 잘 나오지만, 평균이 정확히 3.0이면 그냥 `3`이라고 출력돼요. 같은 여전히 `double`이지만 보여주는 형태만 그런 겁니다.

확인 방법

숫자를 나눠떨어지지 않게 입력해보세요. 예를 들어 1.2, 4를 입력하면 평균이 2.333...이 될 텐데, 이때 `1`처럼 나오는지 확인해보시면 원인이 명확해질 거예요. 여담 소수점까지 잘 나옵니다.

형상 소수점을 보이게 하려면

```
cpp
#include <iomanip> // 추가로 필요

cout << fixed << setprecision(2) << ci.avg << endl;
```

`fixed`를 쓰면 값이 딱 떨어지는 정수여도 강제로 소수점까지 표시합니다. 예를 들어 평균이 3.0이어도 `3.00`으로 출력돼요. `setprecision(2)`는 소수점 이하 자릿수를 2자리로 고정하겠다는 뜻이고, 원하는 자릿수로 바꾸시면 됩니다.

```
cpp
cout << "평균: " << fixed << setprecision(2) << ci.avg << endl;
```

이렇게 바꾸시면 값에 상관없이 항상 소수점까지 나옵니다.

1. vector 사용 시 인덱스 접근 방법 + LLM 사용 이유

vector 사용은 구글링을 통해 손쉽게 예제를 얻을 수 있었다. 그 후 vector를 사용한 배열에 반복문을 사용해 인덱스에 접근하려고 할 때 일반적인 배열과 차이점이 있는지 의문이 생겨 LLM의 도움을 받았다. LLM은 vector를 사용한 배열도 일반적인 배열과 인덱스를 접근하는 방식이 같다고 정보를 제공했고 LLM의 예제를 이해한 뒤 코드로 돌아가 나의 의도에 맞게 코드를 작성했다.

깨달은 점:

LLM에게 정보를 제공받고 내가 한 프롬프트 질문들을 보면 그리 어렵지 않은 질문들을 한 다고 생각되었다. 다음에는 이와 같이 어렵지 않은 질문들은 코드로 먼저 작성을 해보면서 시행착오를 겪고 직접 배우는 것이 더 나의 학습에 도움이 되고 장기 기억으로 남을 확률이 높다고 판단했고 실천으로 옮길 것이다.

2. 소수점 출력 문제 + LLM 사용 이유

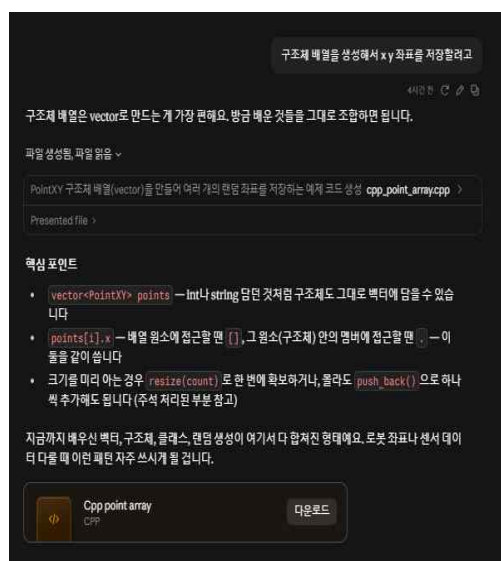
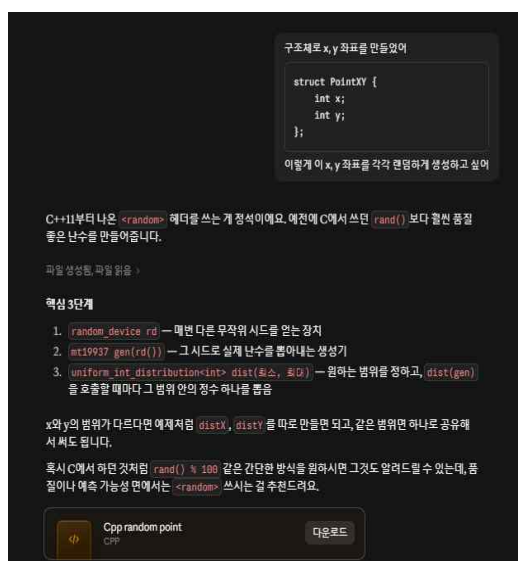
문제에서 평균을 소수점 6번째 자리까지 출력해야한다. 이것을 구현하기 위해 구글링을 통해 `cout << fixed;`, `cout.precision(6);` 같은 강제로 소수점을 몇 번째 까지 출력하게 하는 예제들을 찾아 사용했다. 코드를 실행시켜봤을 때 여전히 정수로 나왔고 이 문제를 해결하기 위해 LLM을 사용했다. LLM에게 문제를 물어봤을 때 평균 변수를 double 실수형으로 선언을 했지만 평균을 구하는 식에서 num과 같은 값들이 정수로 선언되어 있어 정수 계산

후 실수형을 저장되어 소수점이 출력이 되지 않는다고 해결책을 주었다. 그 후 num을 형변환하여 다시 코드를 실행시켜봤을 때는 평균이 소수점까지 잘 출력되었다.

깨달은 점:

LLM을 사용해 문제 원인을 알고보니 너무 쉬운 원인이었다. 이때 당시 이 문제만 해결하면 과제를 완성시킬 수 있다는 마음에 고민하는 시간을 충분히 갖지 않고 LLM을 사용했다. 지금 생각해보면 이러한 행동은 나의 학습에 전혀 도움이 되지 않는다. 다음 과제 시에는 이러한 단순한 오류들은 시간을 충분히 가지면서 스스로 해결할 것이다.

과제 2 AI 사용 내용



1. X, Y 좌표 랜덤하게 생성하는 방법 + LLM 사용 이유

X, Y좌표를 랜덤하게 생성하기 위해서 C++에서는 random 헤더파일을 사용해야 하는 것은 구글링 통해 알게되었다. 하지만 구조체 멤버에 있는 X, Y 좌표들을 랜덤하게 생성하여 각각 저장하는 방법을 알고 싶었고 그 예제와 구현 방법들을 알기 위해서 LLM을 사용했다. LLM은 C언어 rand() 함수를 사용했지만 C++에는 품질 좋은 난수 생성 코드를 알려주었다. 이 난수 생성 코드를 사용하여 x, y좌표를 랜덤하게 생성했고 구조체 멤버에 접근하여 temp.x = distX(gen); temp.y = distY(gen); 같은 코드들로 구조체 멤버 X, Y 좌표를 랜덤하게 생성할 수 있었다.

깨달은 점:

구글링에서는 내가 참고할만한 예제들이 많지 않았다. 그래서 LLM을 사용해 내가 원하는 구현 방법, 예제들을 정확히 입력해 도움을 받았다. LLM으로 코드를 한 번에 다 작성하는 것 보다 공부를 할 땐 코드의 구현 방법, 예제가 필요한 경우에 LLM을 사용한다면 공부를 더 효율적일 수 있겠다고 생각했다.

과제 3 AI 사용 내용



1. 플레이어의 위치와 몬스터의 위치가 동일할 때 공격이 가능한 것과 공격 시 몬스터의 HP가 감소 구현 방법

플레이어 위치와 몬스터의 위치가 동일할 때 공격하는 것은 조건문을 사용해 좌표가 동일할 때 공격하도록 하는 것까지 구현했다. 하지만 공격 함수는 플레이어의 클래스에 선언되어 있고 몬스터의 위치는 몬스터 클래스에 선언되어있어 몬스터의 x, y좌표를 플레이어의 클래스에 어떻게 가져와 사용 해야할지 의문이였다. 이 의문을 해결하기 위해 LLM을 사용했다. LLM은 몬스터 클래스 객체를 만들어 그 객체의 x, y 좌표 변수를 활용하여 문제를 해결하라는 해결책을 제공해주었다.

깨달은 점:

이 문제는 class에 대해 잘 이해하지 못해 발생한 문제이다. 플레이어 클래스에 선언된 함수들을 main문에서 사용할 때는 플레이어 객체를 만들어 함수를 사용했다. 이것은 구글링을 통해 얻은 예제들을 참고해 구현하였다. 함수들을 사용할 때만 클래스 객체를 활용해 클래스에 선언된 변수도 객체로 접근할 수 있다는 것을 인지하지 못하고 있었다. 그래서 몬스터의 x, y 좌표를 객체로 활용을 하지 못했던 것이라고 생각했고 class에 대해 깊은 이해가 필요하다고 깨달았다.